PROJECT KICKOFF · 2026

소형 open-weight LLM 앙상블 기반 취약점 연구 Multi-Agent Orchestration

On-premise 소형 모델 조합과 하네스 엔지니어링을 통한 취약점 탐지 파이프라인

DURATION
2026.08.03 - 2026.10.17

PROGRAM
WHITEHAT SCHOOL 4th

TEAM
화햇대장

CONTENTS

01 화햇대장 팀 소개 ——— 03p

02 프로젝트 개요 ——— 05p

03 프로젝트 세부 계획 ——— 10p

04 프로젝트 설계 ——— 13p

05 초기 하네스 설계 및 구축 ——— 23p

01. 화햇대장 팀 소개

01. 화햇대장 팀 소개

화햇대장은 다양한 경험과 역량을 갖춘 8명의 인원으로 구성되어 있습니다.
각자의 강점을 바탕으로 협력하여 프로젝트의 목표를 달성해 나가겠습니다.

MENTOR 조현우	오윤경 12반 · PM	김하은 12반	손수민 20반	조민규 31반
PL 김진겸	서영채 11반	김예준 18반	김레빈 29반	류한솔 35반

02. 프로젝트 개요

02-1. 프로젝트 정의

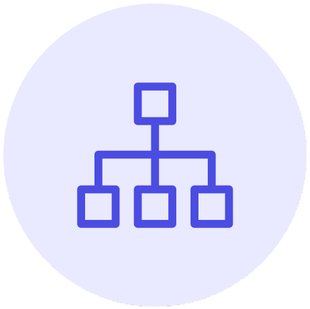
본 과제에서는 **여러 개의 소형 오픈소스 LLM을 협력시키는 소프트웨어 취약점 탐지 자동화 시스템**을 개발합니다.
시스템은 다음의 세 가지 구성 요소로 이루어집니다.

취약점 탐지 자동화 시스템

01. 소형 오픈 모델 앙상블

로컬 환경에서 실행되는
다수의 소형 오픈소스 LLM

02. Orchestration 계층



각 모델에 적합한 작업을 분배하며
전체 탐지 과정을 조율

03. 검증 엔진

모델이 제안한
취약점 후보 검증

? 이 과제가 검증하려는 질문

작은 오픈 모델을 여러 개 묶었을 때
Frontier 모델의 취약점 탐지 성능에
어디까지 근접할 수 있으며,
그 대가로 **비용**과 **프라이버시**에서
얼마나 큰 이점을 얻을 수 있는가?

02-2. 프로젝트 필요성

본 프로젝트가 필요한 이유는 다음과 같습니다.

1. 비용 문제

- 대형 모델 사용 시 비용이 매우 큼
- 권한 관련 취약점 검사 24건 실행에 297~2,970달러, 전체 213건 실행에 2,636~26,359달러 소요

2. 프라이버시 문제

- 상용 API 사용 시 **코드가 외부 서버로 전송**
- 기업의 기밀 정보 유출 가능성 → 보안/규정 위반 소지

3. 평가 기준의 이동

- **CyberGym(2026.07) 평가 규칙 변경:** 성공률뿐 아니라 실행 비용도 함께 보고하도록 변경
- 자체 장비에서 실행한 로컬 모델은 비용 0으로 산정 가능
- 평가 기준이 성능 중심에서 **성능 + 비용 효율성을 함께 고려하는 방향**으로 변화

4. 작은 모델로도 가능하다는 근거

- **IRIS · BACFuzz · Tencent:** 소형 오픈소스 LLM만으로 취약점 탐지 가능성 입증
- **핵심 전략:** 규칙 기반으로 후보를 좁히고, LLM은 제한된 판단만 수행
- **재현 결과:** 후보 132개 → 7개 축소, 최종 4건 정확히 탐지

02-3. 프로젝트 목표

본 프로젝트의 목표는 최고 성능이 아닌
정확도와 비용을 함께 고려한 최적의 Multi-Agent 구조를 설계하는 것입니다.

계산량을 고정하여 다음을 비교합니다.

구분	같은 질문을 여러 번	서로 다른 질문으로 역할 분담
단일 모델	B1	B1'
여러 회사 모델	B3	B3'

이 비교를 통해 다음의 항목을 확인하는 것이 1차 목표입니다.

- 역할을 나눈 효과가 있는지: B1 → B1′ 성능 향상 지표를 통해 측정
- 모델을 섞은 효과가 있는지: B1′ → B3′ 성능 향상 지표를 통해 측정

02-3. 프로젝트 목표

프로젝트의 최종 산출물은 다음과 같습니다.

최종 산출물	목표	평가 기준
Multi-Agent 취약점 탐지 파이프라인	정해진 계산량 안에서 4개 비교 조건 (B1, B1′, B3, B3′)을 모두 실행	• 입력·출력 토큰 분리 기록 • 캐시 재사용률 • 단계별 후보 제거율 기록 • 실패 원인
Frontier 모델 대비 비교 표	비교표의 모든 항목을 실제 측정 결과로 작성	• 공개 기준선(CodeQL 27, GPT-4 55 등) • 취약점 1건당 비용 • 사람 검토 시간 • 외부 전송 데이터량
Zero-Day 취약점	• 기본 목표: 오탐 제거 • 최종 목표: 신규 취약점 제보	• 표본(150건) 기반 오탐률 • AI 판정과 사람 판정의 일치도

발견된 취약점은 최소 2인 이상의 검증을 거친 후 제보하며, AI가 생성한 내용을 그대로 제출하지 않습니다.
또한 공개된 소스코드와 직접 구축한 로컬 환경에서만 연구를 수행하여 관련 규정을 준수합니다.

03. 프로젝트 세부 계획

03. 프로젝트 세부 계획

프로젝트의 세부 계획은 다음과 같습니다. (1/2)

킥오프	1차	2차	중간 발표
8월 3일	8월 17일	8월 31일	9월 5일
프로젝트 기반 마련	**모델 서빙 환경 안정화**	**Multi-Agent 구조 설계 및 앙상블 실험**	**프로젝트 실현 가능성 검증**
• 역할 분담표 작성 • 로컬 서빙 환경 구축 • 후보 소형 모델 조사 • 비교지표 정의서 작성 (가성비/privacy/효과)	• 소형 모델 서빙파이프라인 구축 • 단일 모델 베이스라인 설계	• Multi-Agent Orchestration 구조 완성 • 앙상블 실험 결과 확보 • Harness v1 구축	• 취약점 End-to-End 도출 시연 • Frontier 모델 대비 비교 결과 확보

03. 프로젝트 세부 계획

프로젝트의 세부 계획은 다음과 같습니다. (2/2)

3차

9월 7일

Harness 안정성 및 재현성 확보

- 고도화된 Harness 구축
- Agent 간 통신 안정화

4차

9월 21일

정량적 성능 비교 및 실전 환경 확장

- Frontier 모델 대비 비용·성능·Privacy 벤치마크 결과표 작성
- Zero-day 헌팅

5차

10월 5일

실질적 성과 (제보 가능한 취약점) 확보

- 취약점 제보
- 헌팅 및 벤치마크 종합

최종 발표

10월 17일

성과 공유 및 프로젝트 결과 공개

- 오픈소스 공개
- 비교 리포트/논문 작성
- 최종 발표 자료 작성

04. 프로젝트 설계

04-1. 전체 아키텍처 설계

본 프로젝트는 다양한 분석 도구와 LLM을 연계한 Multi-Agent Vulnerability Harness를 설계하여 펌웨어의 취약점을 분석합니다.
분석 도구가 범위를 축소하고 LLM이 취약점 판단, RCA, 오탐 검토 및 리포트 생성을 수행하는 Orchestration 기반 구조를 목표로 합니다.

구성 요소	도구 기반 처리	LLM 활용
Preprocessor	펌웨어 추출, 일 타입 분류, 바이너리·소스·설정 파일 식별	X
Code / Binary Index	tree-sitter, symbol table, strings, xref, function boundary, call graph 추출	필요한 context 선별
Evidence Store	정적 분석 결과, crash log, sanitizer log, fuzzing 결과 저장	판단 근거 제공
Retriever	symbol, call graph, embedding, reranker 기반 context capsule 생성	검색된 context 기반 분석
Scout	위험 함수, 입력 경로, crash 위치 기반 후보 우선순위 결정	후보 취약점 1차 분류(Triage)
Auditor	source-to-sink, memory lifetime, boundary check, config 영향 분석	취약점 가설 수립 및 RCA 작성
Critic	다른 모델을 활용한 교차 검증	오탐 및 도달 가능성 검토, 가드 확인
Evidence Gate	근거 기준 충족 여부 판단	근거 기반 결과 검증
Tool Validation	build, sanitizer, fuzzing, PoC 실행 로그 기반 검증	검증 결과 해석 및 보완
Reporter	중복 제거 및 템플릿 기반 리포트 생성	리포트 설명 및 RCA 보강

04-2. 병목을 줄이는 핵심 기능

LLM 이전 단계에서 구현할 핵심 기능은 다음과 같습니다.

기능	구현 방식	기대 효과
Codebase Tokenization	tree-sitter 기반 함수·Symbol·AST 단위 분해	저장소 전체를 LLM에 입력하지 않고 필요한 코드만 활용
Binary / Firmware 분석	binwalk, strings, file, readelf, objdump, Ghidra Headless 활용	소스코드가 없는 바이너리·펌웨어도 분석 가능
Call Graph 생성	함수 호출 관계 및 Caller/Callee 추출	분석 범위를 관련 함수 중심으로 축소
위험 패턴 탐지	Semgrep, clang-tidy, grep rule, custom rule 활용	취약점 후보 우선 선별
Crash Triage	Sanitizer log, Stack trace, Crash location 분석	LLM이 분석할 지점을 명확히 지정
Fuzzing 연동	AFL++, libFuzzer 및 Coverage 정보 수집	실행 기반 분석 근거 확보
Evidence Store	SQLite, DuckDB 또는 JSONL 기반 저장	분석 결과 공유 및 중복 분석 방지
Context Capsule 생성	Target function, Caller/Callee, Sink, Log 묶음 구성	LLM 입력 크기 및 토큰 사용량 절감
Budget Ledger	Token, GPU-second, Latency, Cache hit 기록	리소스 사용량 모니터링 및 비용 관리

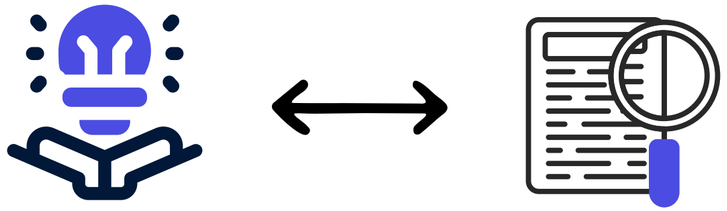
04-3. 단순 RAG의 한계와 Knowledge Base 설계

단순 chunk 기반 RAG로는 코드의 전역 문맥 및 심볼 관계 파악, 근거 추적이 어렵다는 한계점이 있습니다.
따라서 재사용 지식과 세션 관찰 사실을 분리해 신뢰도 높은 참조 구조를 설계하였습니다.

단순 RAG의 한계

1. 관련 없는 문서 검색
2. 정확한 정보 검색의 어려움
3. Context Window 초과
4. Token 비용 및 추론시간 증가

Knowledge Base와 Evidence Store 분리



Knowledge Base
재사용할 수 있는 일반 지식

Evidence Store
Artifact 분석 결과

Knowledge Base의 데이터 소스는 CWE/CVE, 패치 정보, CTF Write-up, 도구 사용 방법 등의 공개 지식입니다.
일반 지식과 분석 대상의 실제 관찰 결과를 분리하여 관리하고 취약점 판단은 Evidence를 중심으로 수행합니다.

04-4. 계층형 Knowledge Base 구조 및 Hybrid Retrieval

데이터를 특성에 맞게 계층적으로 저장하고 적합한 검색 방식을 선택하여 필요한 Context만 제공합니다.

계층형 Knowledge Base 구조

- **L0** Metadata Layer — 파일 경로, 함수명, symbol, binary offset, firmware path, artifact hash
- **L1** Summary Layer — CVE 요약, CWE 설명, finding 요약, 패치 요약
- **L2** Evidence Layer — crash log, sanitizer log, fuzzing result, static analyzer output
- **L3** Graph Layer — caller/callee, xref, source-to-sink, config dependency
- **L4** Detail Layer — 함수 코드, 디컴파일 결과, 패치 hunk, 원본 write-up
- **L5** Detail Layer — 최종 finding, RCA, false positive 사유

Hybrid Retrieval 방식을 이용하여 검색 대상의 특성에 따라 Exact Search, FTS, Embedding Search, Graph Retrieval을 조합함으로써 불필요한 Context를 줄이고 필요한 Evidence를 효과적으로 검색합니다.

04-5. Context Management 전략

Context Management는 필요한 Evidence만 선별하여 LLM에 단계적으로 제공하는 전략입니다.
다음과 같은 방법으로 Context를 효율적으로 관리합니다.

1. Progressive Context Expansion

- 최소 Context부터 제공
- 필요 시 관련 Evidence 단계적 확장
- 불필요한 Token 사용 최소화

2. Context Capsule

- 후보별 최소 분석 Context 구성
- Target Function 중심 정보 제공
- Evidence 및 원본 위치 포함

3. 중복 관리와 Cache

- 동일 Artifact 분석 결과 재사용
- Context Capsule 및 Finding Cache 활용
- Artifact 변경 시 재검증 수행

4. Context Budget

- Context 크기 및 검색 범위 제한
- 필요한 Evidence만 선별 제공
- Token 비용과 추론 시간 절감

04-6. 앙상블 및 Orchestration 전략

초기에는 Scout → Auditor → Critic → Evidence Gate → Tool Validation → Report 순으로 단계별 분석을 수행합니다.
각 Agent는 이전 단계의 결과를 기반으로 취약점 후보를 선별·검증하며, 최종적으로 근거 기반 리포트를 생성합니다.

단계	담당	역할
1차 수집	ToolRunner	위험 함수, Crash, 정적 분석 결과 수집
2차 선별	Scout	취약점 후보 선별 및 우선순위 산정
3차 분석	Auditor	취약점 가설 수립 및 Source-to-Sink 분석
4차 검증	Critic	오탐 가능성, 도달 가능성, Guard 조건 검토
5차 승인	Evidence Gate	근거 충족 여부 및 Finding 승격 판단
6차 검증	Tool Validation	Fuzzing, Sanitizer, Build 결과 기반 검증
7차 검증	Report	중복 제거 및 RCA 중심 리포트 생성

각 Agent는 `role, model_profile, tool_policy, token_budget, output_schema`로 구성된 실행 계약을 기반으로 동작합니다.
이를 통해 특정 LLM에 종속되지 않고 동일한 역할에 여러 모델을 적용하여 성능을 비교·평가할 수 있습니다.

04-7. 평가 및 벤치마크 설계

정답이 알려진 취약 샘플과 정상 샘플을 활용하여
취약점 탐지 성능, 근거 품질, 오탐률, 그리고 LLM 호출 비용을 종합적으로 평가합니다.

설계한 벤치마크의 구성은 다음과 같습니다.

조건	구성	비교 목적
B1	단일 LLM 또는 단순 Retrieval 기반 소스코드 분석	Source-only 접근의 한계 확인
B1′	바이너리·펌웨어 Artifact를 포함한 분석	입력 범위 확장 효과 검증
B3	Multi-Agent Orchestration(Scout, Auditor, Critic) 적용	역할 분리 및 모델 협업 효과 검증
B3′	B3 + Evidence Gate + Tool Validation	오탐 감소 및 근거 품질 향상 검증

04-7. 평가 및 벤치마크 설계

설계한 평가 데이터의 구성은 다음과 같습니다.

평가 데이터	설명
CTF 시스템 해킹 문제	BOF, UAF, Command injection 등 명확한 정답을 가진 샘플
Juliet Test Suite	C/C++ 기반 취약 코드와 정상 코드 쌍
공개 CVE 패치 샘플	취약 버전과 패치 버전
공개 펌웨어 샘플	실제 펌웨어 구조
정상 샘플(Benign)	알려진 취약점이 없는 Benign code/Binary/Firmware
Fuzzing Crash 샘플	Sanitizer 또는 Crash log가 존재하는 샘플

평가에는 취약 샘플과 정상 샘플을 모두 포함하여 탐지 성능과 오탐률을 함께 측정합니다. 특히 패치된 버전, 안전한 API 사용 예제, Crash가 발생하지 않는 펌웨어 컴포넌트를 포함하여 단순히 위험 함수의 존재만으로 취약점으로 판단하지 않는지 검증합니다.

04-8. 데이터 누출 방지 기준

평가의 신뢰도를 확보하기 위해 학습, 검색(RAG), 평가 데이터를 명확히 분리합니다.

기준	설명
Train/Test 분리	KB와 평가 데이터 간 중복 및 정답 누출 방지
CVE ID Masking	CVE 번호, 문제명, Flag, Write-up 제목 등 정답 단서 제거
Patch Leakage 방지	평가 대상의 Patch Diff가 Prompt 또는 Retrieval Context에 포함되지 않도록 제한
중복 데이터 제거	동일 코드·Challenge·Firmware Version의 Train/Test 중복 방지
Time Split	특정 시점을 기준으로 Knowledge Base와 평가 데이터를 분리
Prompt Logging	모델 입력 Context를 JSONL/OpenTelemetry Trace로 기록
Reproducibility	동일 Artifact·Model·Retrieval 조건에서 결과 재현 보장

MVP의 목표는 완전한 Zero-day 취약점 자동 탐지가 아니라
취약점 후보 수집, 우선순위화, RCA 초안 작성, 근거 정리, 오탐 검토를 자동화하여 분석 시간을 단축할 수 있는지 검증하는 것입니다.

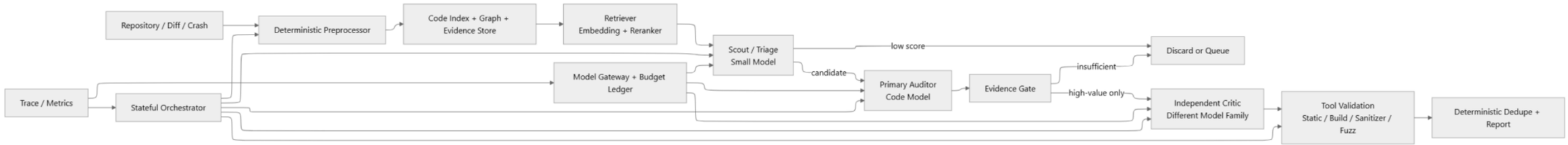
05. 초기 하네스 설계 및 구축

05-1. 공개 시스템에서 가져올 설계 요소

초기 목표는 적은 수의 역할을 계층적으로 실행하여, 제한된 연산 예산 안에서 검증 가능한 finding을 최대화하는 것입니다.

참고할 시스템	참고할 점	차이점
Microsoft MDASH	• Prepare → Scan → Validate/Debate → Dedupe → Prove의 단계 분리 • 서로 다른 모델의 교차 검증 • evidence 중심 confidence 상승	100개 이상의 에이전트와 frontier model 중심 운영 규모
Theori RoboDuck	• 역할별 모델 매핑 • 정적 분석·퍼징·LLM 분석의 병렬 결합 • crash triage → PoV → patch 흐름 • agent trace와 호출 비용 관찰	AIxCC 결선의 대규모 API 예산에 맞춘 무제한형 agent fan-out
Buttercup / AIxCC CRS	Orchestrator, seed generation, fuzzing, program model, patcher의 역할을 분리하고 LLM을 기존 분석 도구의 대체재가 아닌 증폭기로 사용	초기 단계부터 Kubernetes와 다수의 분산 서비스로 구성됨에 따른 운영 복잡도
Atlantis / OSS-CRS	• 표준 CLI, Docker 기반 재현 • 자원·LLM budget 설정 • 모듈별 교체와 ensemble 실행 계약	AIxCC 제출 규격과 patch 생성까지 모두 포함하는 전체 CRS 범위

05-2. 초기 하네스 구조



Agent 설계 원칙

Agent를 서비스가 아닌 실행 계약으로 정의

Role	Scout, Auditor, Critic 등 수행 목적
Model Profile	사용할 모델 및 fallback
Tool Policy	사용 가능한 분석 도구와 명령
Token Budget	입력·출력·재시도 한도
Output Schema	다음 Agent가 소비할 구조화 결과

초기 Multi-Agent 구조

초기에는 4개의 핵심 Agent로 구성

Agent	주요 역할
Scout / Triage	분석 우선순위 선정 및 후보 필터링
Primary Auditor	취약점 분석 및 Evidence 생성
Independent Critic	다른 모델을 활용해 상위 후보에 대한 독립적 재검증 진행
Evidence Judge	정적 분석·빌드·Fuzzing 결과를 종합하여 최종 판정

05-3. 초기 모델 구성

Code retrieval

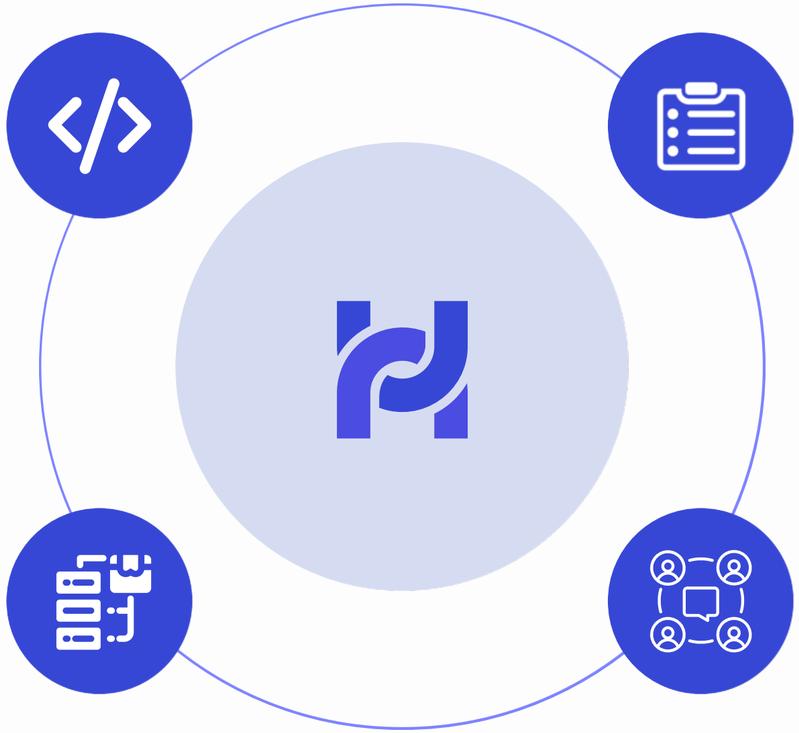
Qwen3-Embedding-0.6B
Qwen3-Reranker-0.6B

- 저비용으로 후보 문맥을 검색·축소
- 상시 구동 (CPU 또는 소량 GPU)

Scout / Triage

Qwen3-8B

- 후보 취약점 분류, 요약, 우선순위 결정
- 빠른 1차 필터링 수행
- 짧은 출력, thinking 최소화, 높은 병렬성



Primary Auditor

Qwen3-Coder-30B-A3B-Instruct

- 30.5B (3.3B active) MoE, 256K Context
- Repository-scale 분석에 최적화
- 선별 후보에 호출
- 양자화 및 context 상한 검증 후 운영

Independent Critic

Devstral Small 2 24B

- 다른 모델 계열을 활용한 독립 검증
- Codebase 탐색 및 Tool-Use 지원
- 상위 후보만 검증하여 비용 최적화

05-4. Cost-Effective 및 토큰 효율 전략

1. Tool-first, Model-second

- 기존 분석 도구 우선 활용
- Diff · Static Analysis · Fuzzing 우선 수행
- LLM은 필요한 코드만 분석

2. Full repository prompt 금지

- 필요한 문맥만 전달
- Target Function 중심 Context
- Caller/Callee · Diff · Evidence 포함

3. 두 단계의 Retrieval 구성

- symbol · call graph · keyword 기반으로 넓게 탐색
- Embedding + Reranker로 상위 context 재선별

4. 구조화된 정보 전달

Agent 간 구조화된 정보 전달
(Candidate, Evidence, Decision, MissingEvidence 구조만 전달)

5. Prefix cache와 batch 활용

- 공통 Prompt를 Prefix로 고정
- 반복 요청의 Prefill 비용 절감
- Batch 처리로 추론 효율 향상

6. 비용 기반 성능 관리

- Token · GPU 사용량
- Cache Hit Ratio
- Finding당 비용(GPU-hour)

감사합니다.